12 essays, 33,617 words. all of them are at paulgraham.com, free, as they should be. this is one reader's copy, printed at home to be read away from a screen, with a pencil.

made with on-paper. github.com/workinprogmess/on-paper

01

taste for makers

february 2002
paulgraham.com/taste.html

saying that taste is just personal preference is a good way to prevent disputes. the trouble is, it's not true. you feel this when you start to design things.

whatever job people do, they naturally want to do better. football players like to win games. ceos like to increase earnings. it's a matter of pride, and a real pleasure, to get better at your job. but if your job is to design things, and there is no such thing as beauty, then there is *no way to get better at your job.* if taste is just personal preference, then everyone's is already perfect: you like whatever you like, and that's it.

as in any job, as you continue to design things, you'll get better at it. your tastes will change. and, like anyone who gets better at their job, you'll know you're getting better. if so, your old tastes were not merely different, but worse. poof goes the axiom that taste can't be wrong.

relativism is fashionable at the moment, and that may hamper you from thinking about taste, even as yours grows. but if you come out of the closet and admit, at least to yourself, that there is such a thing as good and bad design, then you can start to study good design in detail. how has your taste changed? when you made mistakes, what caused you to make them? what have other people learned about design?

once you start to examine the question, it's surprising how much different fields' ideas of beauty have in common. the same principles of good design crop up again and again.

good design is simple. you hear this from math to painting. in math it means that a shorter proof tends to be a better one. where axioms are concerned, especially, less is more. it means much the same thing in programming. for architects and designers it means that beauty should depend on a few carefully chosen structural elements rather than a profusion of superficial ornament. (ornament is not in itself bad, only when it's camouflage on insipid form.) similarly, in painting, a still life of a few carefully observed and solidly modelled objects will tend to be more interesting than a stretch of flashy but mindlessly repetitive painting of, say, a lace collar. in writing it means: say what you mean and say it briefly.

it seems strange to have to emphasize simplicity. you'd think simple would be the default. ornate is more work. but something seems to come over people when they try to be creative. beginning writers adopt a pompous tone that doesn't sound anything like the way they speak. designers trying to be artistic resort to swooshes and curlicues. painters discover that they're expressionists. it's all evasion. underneath the long words or the "expressive" brush strokes, there is not much going on, and that's frightening.

when you're forced to be simple, you're forced to face the real problem. when you can't deliver ornament, you have to deliver substance.

good design is timeless. in math, every proof is timeless unless it contains a mistake. so what does hardy mean when he says there is no permanent place for ugly mathematics? he means the same thing kelly johnson did: if something is ugly, it can't be the best solution. there must be a better one, and eventually someone will discover it.

aiming at timelessness is a way to make yourself find the best answer: if you can imagine someone surpassing you, you should do it yourself. some of the greatest masters did this so well that they left little room for those who came after. every engraver since durer has had to live in his shadow.

aiming at timelessness is also a way to evade the grip of fashion. fashions almost by definition change with time, so if you can make something that will still look good far into the future, then its appeal

hard problems call for great efforts. in math, difficult proofs require ingenious solutions, and those tend to be interesting. ditto in engineering.

when you have to climb a mountain you toss everything unnecessary out of your pack. and so an architect who has to build on a difficult site, or a small budget, will find that he is forced to produce an elegant design. fashions and flourishes get knocked aside by the difficult business of solving the problem at all.

not every kind of hard is good. there is good pain and bad pain. you want the kind of pain you get from going running, not the kind you get from stepping on a nail. a difficult problem could be good for a designer, but a fickle client or unreliable materials would not be.

in art, the highest place has traditionally been given to paintings of people. there is something to this tradition, and not just because pictures of faces get to press buttons in our brains that other pictures don't. we are so good at looking at faces that we force anyone who draws them to work hard to satisfy us. if you draw a tree and you change the angle of a branch five degrees, no one will know. when you change the angle of someone's eye five degrees, people notice.

when bauhaus designers adopted sullivan's "form follows function," what they meant was, form *should* follow function. and if function is hard enough, form is forced to follow it, because there is no effort to spare for error. wild animals are beautiful because they have hard lives.

good design looks easy. like great athletes, great designers make it look easy. mostly this is an illusion. the easy, conversational tone of good writing comes only on the eighth rewrite.

in science and engineering, some of the greatest discoveries seem so simple that you say to yourself, i could have thought of that. the discoverer is entitled to reply, why didn't you?

some leonardo heads are just a few lines. you look at them and you think, all you have to do is get eight or ten lines in the right place and you've made this beautiful portrait. well, yes, but you have to get them in *exactly* the right place. the slightest error will make the whole thing collapse.

line drawings are in fact the most difficult visual medium, because they demand near perfection. in math terms, they are a closed-form solution; lesser artists literally solve the same problems by successive approximation. one of the reasons kids give up drawing at ten or so is that they decide to start drawing like grownups, and one of the first things they try is a line drawing of a face. smack!

in most fields the appearance of ease seems to come with practice. perhaps what practice does is train your unconscious mind to handle tasks that used to require conscious thought. in some cases you literally train your body. an expert pianist can play notes faster than the brain can send signals to his hand. likewise an artist, after a while, can make visual perception flow in through his eye and out through his hand as automatically as someone tapping his foot to a beat.

when people talk about being in "the zone," i think what they mean is that the spinal cord has the situation under control. your spinal cord is less hesitant, and it frees conscious thought for the hard problems.

good design uses symmetry. i think symmetry may just be one way to achieve simplicity, but it's important enough to be mentioned on its own. nature uses it a lot, which is a good sign.

there are two kinds of symmetry, repetition and recursion. recursion means repetition in subelements, like the pattern of veins in a leaf.

it helps to have a medium that makes change easy. when oil paint replaced tempera in the fifteenth century, it helped painters to deal with difficult subjects like the human figure because, unlike tempera, oil can be blended and overpainted.

good design can copy. attitudes to copying often make a round trip. a novice imitates without knowing it; next he tries consciously to be original; finally, he decides it's more important to be right than original.

unknowing imitation is almost a recipe for bad design. if you don't know where your ideas are coming from, you're probably imitating an imitator. raphael so pervaded mid-nineteenth century taste that almost anyone who tried to draw was imitating him, often at several removes. it was this, more than raphael's own work, that bothered the pre-raphaelites.

the ambitious are not content to imitate. the second phase in the growth of taste is a conscious attempt at originality.

i think the greatest masters go on to achieve a kind of selflessness. they just want to get the right answer, and if part of the right answer has already been discovered by someone else, that's no reason not to use it. they're confident enough to take from anyone without feeling that their own vision will be lost in the process.

good design is often strange. some of the very best work has an uncanny quality: euler's formula, bruegel's *hunters in the snow,* the sr-71, lisp. they're not just beautiful, but strangely beautiful.

i'm not sure why. it may just be my own stupidity. a can-opener must seem miraculous to a dog. maybe if i were smart enough it would seem the most natural thing in the world that $e^{i*pi} = -1$. it is after all necessarily true.

most of the qualities i've mentioned are things that can be cultivated, but i don't think it works to cultivate strangeness. the best you can do is not squash it if it starts to appear. einstein didn't try to make relativity strange. he tried to make it true, and the truth turned out to be strange.

at an art school where i once studied, the students wanted most of all to develop a personal style. but if you just try to make good things, you'll inevitably do it in a distinctive way, just as each person walks in a distinctive way. michelangelo was not trying to paint like michelangelo. he was just trying to paint well; he couldn't help painting like michelangelo.

the only style worth having is the one you can't help. and this is especially true for strangeness. there is no shortcut to it. the northwest passage that the mannerists, the romantics, and two generations of american high school students have searched for does not seem to exist. the only way to get there is to go through good and come out the other side.

good design happens in chunks. the inhabitants of fifteenth century florence included brunelleschi, ghiberti, donatello, masaccio, filippo lippi, fra angelico, verrocchio, botticelli, leonardo, and michelangelo. milan at the time was as big as florence. how many fifteenth century milanese artists can you name?

something was happening in florence in the fifteenth century. and it can't have been heredity, because it isn't happening now. you have to assume that whatever inborn ability leonardo and michelangelo had, there were people born in milan with just as much. what happened to the milanese leonardo?

there are roughly a thousand times as many people alive in the us right now as lived in florence during the fifteenth century. a thousand leonardos and a thousand michelangelos walk among us. if dna ruled,

02

hackers and painters

may 2003
paulgraham.com/hp.html

(this essay is derived from a guest lecture at harvard, which incorporated an earlier talk at northeastern.)

when i finished grad school in computer science i went to art school to study painting. a lot of people seemed surprised that someone interested in computers would also be interested in painting. they seemed to think that hacking and painting were very different kinds of work-- that hacking was cold, precise, and methodical, and that painting was the frenzied expression of some primal urge.

both of these images are wrong. hacking and painting have a lot in common. in fact, of all the different types of people i've known, hackers and painters are among the most alike.

what hackers and painters have in common is that they're both makers. along with composers, architects, and writers, what hackers and painters are trying to do is make good things. they're not doing research per se, though if in the course of trying to make good things they discover some new technique, so much the better.

i've never liked the term "computer science." the main reason i don't like it is that there's no such thing. computer science is a grab bag of tenuously related areas thrown together by an accident of history, like yugoslavia. at one end you have people who are really mathematicians, but call what they're doing computer science so they can get darpa grants. in the middle you have people working on something like the natural history of computers-- studying the behavior of algorithms for routing data through networks, for example. and then at the other extreme you have the hackers, who are trying to write interesting software, and for whom computers are just a medium of expression, as concrete is for architects or paint for painters. it's as if mathematicians, physicists, and architects all had to be in the same department.

sometimes what the hackers do is called "software engineering," but this term is just as misleading. good software designers are no more engineers than architects are. the border between architecture and engineering is not sharply defined, but it's there. it falls between what and how: architects decide what to do, and engineers figure out how to do it.

what and how should not be kept too separate. you're asking for trouble if you try to decide what to do without understanding how to do it. but hacking can certainly be more than just deciding how to implem-

there are worse things than having people misunderstand your work. a worse danger is that you will yourself misunderstand your work. related fields are where you go looking for ideas. if you find yourself in the computer science department, there is a natural temptation to believe, for example, that hacking is the applied version of what theoretical computer science is the theory of. all the time i was in graduate school i had an uncomfortable feeling in the back of my mind that i ought to know more theory, and that it was very remiss of me to have forgotten all that stuff within three weeks of the final exam.

now i realize i was mistaken. hackers need to understand the theory of computation about as much as painters need to understand paint chemistry. you need to know how to calculate time and space complexity and about turing completeness. you might also want to remember at least the concept of a state machine, in case you have to write a parser or a regular expression library. painters in fact have to remember a good deal more about paint chemistry than that.

i've found that the best sources of ideas are not the other fields that have the word "computer" in their names, but the other fields inhabited by makers. painting has been a much richer source of ideas than the theory of computation.

for example, i was taught in college that one ought to figure out a program completely on paper before even going near a computer. i found that i did not program this way. i found that i liked to program sitting in front of a computer, not a piece of paper. worse still, instead of patiently writing out a complete program and assuring myself it was correct, i tended to just spew out code that was hopelessly broken, and gradually beat it into shape. debugging, i was taught, was a kind of final pass where you caught typos and oversights. the way i worked, it seemed like programming consisted of debugging.

for a long time i felt bad about this, just as i once felt bad that i didn't hold my pencil the way they taught me to in elementary school. if i had only looked over at the other makers, the painters or the architects, i would have realized that there was a name for what i was doing: sketching. as far as i can tell, the way they taught me to program in college was all wrong. you should figure out programs as you're writing them, just as writers and painters and architects do.

realizing this has real implications for software design. it means that a programming language should, above all, be malleable. a programming language is for thinking of programs, not for expressing programs you've already thought of. it should be a pencil, not a pen. static typing would be a fine idea if people actually did write programs the way they taught me to in college. but that's not how any of the hackers i know write programs. we need a language that lets us scribble and smudge and smear, not a language where you have to sit with a teacup of types balanced on your knee and make polite conversation with a strict old aunt of a compiler.

while we're on the subject of static typing, identifying with the makers will save us from another problem that afflicts the sciences: math envy. everyone in the sciences secretly believes that mathematicians are smarter than they are. i think mathematicians also believe this. at any rate, the result is that scientists tend to make their work look as mathematical as possible. in a field like physics this probably doesn't do much harm, but the further you get from the natural sciences, the more of a problem it becomes.

a page of formulas just looks so impressive. (tip: for extra impressiveness, use greek variables.) and so there is a great temptation to work on problems you can treat formally, rather than problems that are, say, important.

if hackers identified with other makers, like writers and painters, they wouldn't feel tempted to do this. writers and painters don't suffer from math envy. they feel as if they're doing something completely unrelated. so are hackers, i think.

i think the answer to this problem, in the case of software, is a concept known to nearly all makers: the day job. this phrase began with musicians, who perform at night. more generally, it means that you have one kind of work you do for money, and another for love.

nearly all makers have day jobs early in their careers. painters and writers notoriously do. if you're lucky you can get a day job that's closely related to your real work. musicians often seem to work in record stores. a hacker working on some programming language or operating system might likewise be able to get a day job using it.[1]

when i say that the answer is for hackers to have day jobs, and work on beautiful software on the side, i'm not proposing this as a new idea. this is what open-source hacking is all about. what i'm saying is that open-source is probably the right model, because it has been independently confirmed by all the other makers.

it seems surprising to me that any employer would be reluctant to let hackers work on open-source projects. at viaweb, we would have been reluctant to hire anyone who didn't. when we interviewed programmers, the main thing we cared about was what kind of software they wrote in their spare time. you can't do anything really well unless you love it, and if you love to hack you'll inevitably be working on projects of your own.[2]

because hackers are makers rather than scientists, the right place to look for metaphors is not in the sciences, but among other kinds of makers. what else can painting teach us about hacking?

one thing we can learn, or at least confirm, from the example of painting is how to learn to hack. you learn to paint mostly by doing it. ditto for hacking. most hackers don't learn to hack by taking college courses in programming. they learn to hack by writing programs of their own at age thirteen. even in college classes, you learn to hack mostly by hacking.[3]

because painters leave a trail of work behind them, you can watch them learn by doing. if you look at the work of a painter in chronological order, you'll find that each painting builds on things that have been learned in previous ones. when there's something in a painting that works very well, you can usually find version 1 of it in a smaller form in some earlier painting.

i think most makers work this way. writers and architects seem to as well. maybe it would be good for hackers to act more like painters, and regularly start over from scratch, instead of continuing to work for years on one project, and trying to incorporate all their later ideas as revisions.

the fact that hackers learn to hack by doing it is another sign of how different hacking is from the sciences. scientists don't learn science by doing it, but by doing labs and problem sets. scientists start out doing work that's perfect, in the sense that they're just trying to reproduce work someone else has already done for them. eventually, they get to the point where they can do original work. whereas hackers, from the start, are doing original work; it's just very bad. so hackers start original, and get good, and scientists start good, and get original.

the other way makers learn is from examples. for a painter, a museum is a reference library of techniques. for hundreds of years it has been part of the traditional education of painters to copy the works of the great masters, because copying forces you to look closely at the way a painting is made.

writers do this too. benjamin franklin learned to write by summarizing the points in the essays of addison and steele and then trying to reproduce them. raymond chandler did the same thing with detective stories.

hackers, likewise, can learn to program by looking at good programs-- not just at what they do, but the source code too. one of the less publicized benefits of the open-source movement is that it has made it easier

sometimes to avoid stalling. backing off can likewise prevent ambition from stalling. in both painting and hacking there are some tasks that are terrifyingly ambitious, and others that are comfortingly routine. it's a good idea to save some easy tasks for moments when you would otherwise stall.

in hacking, this can literally mean saving up bugs. i like debugging: it's the one time that hacking is as straightforward as people think it is. you have a totally constrained problem, and all you have to do is solve it. your program is supposed to do x. instead it does y. where does it go wrong? you know you're going to win in the end. it's as relaxing as painting a wall.

the example of painting can teach us not only how to manage our own work, but how to work together. a lot of the great art of the past is the work of multiple hands, though there may only be one name on the wall next to it in the museum. leonardo was an apprentice in the workshop of verrocchio and painted one of the angels in his baptism of christ. this sort of thing was the rule, not the exception. michelangelo was considered especially dedicated for insisting on painting all the figures on the ceiling of the sistine chapel himself.

as far as i know, when painters worked together on a painting, they never worked on the same parts. it was common for the master to paint the principal figures and for assistants to paint the others and the background. but you never had one guy painting over the work of another.

i think this is the right model for collaboration in software too. don't push it too far. when a piece of code is being hacked by three or four different people, no one of whom really owns it, it will end up being like a common-room. it will tend to feel bleak and abandoned, and accumulate cruft. the right way to collaborate, i think, is to divide projects into sharply defined modules, each with a definite owner, and with interfaces between them that are as carefully designed and, if possible, as articulated as programming languages.

like painting, most software is intended for a human audience. and so hackers, like painters, must have empathy to do really great work. you have to be able to see things from the user's point of view.

when i was a kid i was always being told to look at things from someone else's point of view. what this always meant in practice was to do what someone else wanted, instead of what i wanted. this of course gave empathy a bad name, and i made a point of not cultivating it.

boy, was i wrong. it turns out that looking at things from other people's point of view is practically the secret of success. it doesn't necessarily mean being self-sacrificing. far from it. understanding how someone else sees things doesn't imply that you'll act in his interest; in some situations-- in war, for example-- you want to do exactly the opposite.[4]

most makers make things for a human audience. and to engage an audience you have to understand what they need. nearly all the greatest paintings are paintings of people, for example, because people are what people are interested in.

empathy is probably the single most important difference between a good hacker and a great one. some hackers are quite smart, but when it comes to empathy are practically solipsists. it's hard for such people to design great software[5]*, because they can't see things from the user's point of view.*

one way to tell how good people are at empathy is to watch them explain a technical question to someone without a technical background. we probably all know people who, though otherwise smart, are just comically bad at this. if someone asks them at a dinner party what a programming language is, they'll say something like ``oh, a high-level language is what the compiler uses as input to generate object code."

notes

[1] the greatest damage that photography has done to painting may be the fact that it killed the best day job. most of the great painters in history supported themselves by painting portraits.

[2] i've been told that microsoft discourages employees from contributing to open-source projects, even in their spare time. but so many of the best hackers work on open-source projects now that the main effect of this policy may be to ensure that they won't be able to hire any first-rate programmers.

[3] what you learn about programming in college is much like what you learn about books or clothes or dating: what bad taste you had in high school.

[4] here's an example of applied empathy. at viaweb, if we couldn't decide between two alternatives, we'd ask, what would our competitors hate most? at one point a competitor added a feature to their software that was basically useless, but since it was one of few they had that we didn't, they made much of it in the trade press. we could have tried to explain that the feature was useless, but we decided it would annoy our competitor more if we just implemented it ourselves, so we hacked together our own version that afternoon.

[5] except text editors and compilers. hackers don't need empathy to design these, because they are themselves typical users.

[6] well, almost. they overshot the available ram somewhat, causing much inconvenient disk swapping, but this could be fixed within a few months by buying an additional disk drive.

[7] the way to make programs easy to read is not to stuff them with comments. i would take abelson and sussman's quote a step further. programming languages should be designed to express algorithms, and only incidentally to tell computers how to execute them. a good programming language ought to be better for explaining software than english. you should only need comments when there is some kind of kludge you need to warn readers about, just as on a road there are only arrows on parts with unexpectedly sharp curves.

thanks *to trevor blackwell, robert morris, dan giffin, and lisa randall for reading drafts of this, and to henry leitner and larry finkelstein for inviting me to speak.*

03

design and research

january 2003
paulgraham.com/desres.html

you're most likely to get good design if the intended users include the designer himself. when you design something for a group that doesn't include you, it tends to be for people you consider to be less sophisticated than you, not more sophisticated.

that's a problem, because looking down on the user, however benevolently, seems inevitably to corrupt the designer. i suspect that very few housing projects in the us were designed by architects who expected to live in them. you can see the same thing in programming languages. c, lisp, and smalltalk were created for their own designers to use. cobol, ada, and java, were created for other people to use.

if you think you're designing something for idiots, the odds are that you're not designing something good, even for idiots.

even if you're designing something for the most sophisticated users, though, you're still designing for humans. it's different in research. in math you don't choose abstractions because they're easy for humans to understand; you choose whichever make the proof shorter. i think this is true for the sciences generally. scientific ideas are not meant to be ergonomic.

over in the arts, things are very different. design is all about people. the human body is a strange thing, but when you're designing a chair, that's what you're designing for, and there's no way around it. all the arts have to pander to the interests and limitations of humans. in painting, for example, all other things being equal a painting with people in it will be more interesting than one without. it is not merely an accident of history that the great paintings of the renaissance are all full of people. if they hadn't been, painting as a medium wouldn't have the prestige that it does.

like it or not, programming languages are also for people, and i suspect the human brain is just as lumpy and idiosyncratic as the human body. some ideas are easy for people to grasp and some aren't. for example, we seem to have a very limited capacity for dealing with detail. it's this fact that makes programing languages a good idea in the first place; if we could handle the detail, we could just program in machine language.

remember, too, that languages are not primarily a form for finished programs, but something that programs have to be developed in. anyone in the arts could tell you that you might want different mediums for the two situations. marble, for example, is a nice, durable medium for finished ideas, but a hopelessly inflexible one for developing new ideas.

a program, like a proof, is a pruned version of a tree that in the past has had false starts branching off all over it. so the test of a language is not simply how clean the finished program looks in it, but how clean the path to the finished program was. a design choice that gives you elegant finished programs may not give you an elegant design process. for example, i've written a few macro-defining macros full of nested backquotes that look now like little gems, but writing them took hours of the ugliest trial and error, and frankly, i'm still not entirely sure they're correct.

we often act as if the test of a language were how good finished programs look in it. it seems so convincing when you see the same program written in two languages, and one version is much shorter. when you approach the problem from the direction of the arts, you're less likely to depend on this sort of test. you don't want to end up with a programming language like marble.

for example, it is a huge win in developing software to have an interactive toplevel, what in lisp is called a read-eval-print loop. and when you have one this has real effects on the design of the language. it would not work well for a language where you have to declare variables before using them, for example. when you're just typing expressions into the toplevel, you want to be able to set x to some value and then start doing things to x. you don't want to have to declare the type of x first. you may dispute either of the

particularly in oil painting. most painters start with a blurry sketch and gradually refine it. if you work this way, then in principle you never have to end the day with something that actually looks unfinished. indeed, there is even a saying among painters: "a painting is never finished, you just stop working on it." this idea will be familiar to anyone who has worked on software.

morale is another reason that it's hard to design something for an unsophisticated user. it's hard to stay interested in something you don't like yourself. to make something good, you have to be thinking, "wow, this is really great," not "what a piece of shit; those fools will love it."

design means making things for humans. but it's not just the user who's human. the designer is human too.

notice all this time i've been talking about "the designer." design usually has to be under the control of a single person to be any good. and yet it seems to be possible for several people to collaborate on a research project. this seems to me one of the most interesting differences between research and design.

there have been famous instances of collaboration in the arts, but most of them seem to have been cases of molecular bonding rather than nuclear fusion. in an opera it's common for one person to write the libretto and another to write the music. and during the renaissance, journeymen from northern europe were often employed to do the landscapes in the backgrounds of italian paintings. but these aren't true collaborations. they're more like examples of robert frost's "good fences make good neighbors." you can stick instances of good design together, but within each individual project, one person has to be in control.

i'm not saying that good design requires that one person think of everything. there's nothing more valuable than the advice of someone whose judgement you trust. but after the talking is done, the decision about what to do has to rest with one person.

why is it that research can be done by collaborators and design can't? this is an interesting question. i don't know the answer. perhaps, if design and research converge, the best research is also good design, and in fact can't be done by collaborators. a lot of the most famous scientists seem to have worked alone. but i don't know enough to say whether there is a pattern here. it could be simply that many famous scientists worked when collaboration was less common.

whatever the story is in the sciences, true collaboration seems to be vanishingly rare in the arts. design by committee is a synonym for bad design. why is that so? is there some way to beat this limitation?

i'm inclined to think there isn't-- that good design requires a dictator. one reason is that good design has to be all of a piece. design is not just for humans, but for individual humans. if a design represents an idea that fits in one person's head, then the idea will fit in the user's head too.

related:

04

six principles for making new things

february 2008
paulgraham.com/newthings.html

as with contrarian investment strategies, that's exactly the point. this technique is successful (in the long term) because it gives you all the advantages other people forgo by trying to seem legit. if you work on overlooked problems, you're more likely to discover new things, because you have less competition. if you deliver solutions informally, you (a) save all the effort you would have had to expend to make them look impressive, and (b) avoid the danger of fooling yourself as well as your audience. and if you release a crude version 1 then iterate, your solution can benefit from the imagination of nature, which, as feynman pointed out, is more powerful than your own.

in the case of viaweb, the simple solution was to make the software run on the server. the overlooked problem was to generate web sites automatically; in 1995, online stores were all made by hand by human designers, but we knew this wouldn't scale. the part that actually mattered was graphic design, not transaction processing. the informal delivery mechanism was me, showing up in jeans and a t-shirt at some retailer's office. and the crude version 1 was, if i remember correctly, less than 10,000 lines of code when we launched.

the power of this technique extends beyond startups and programming languages and essays. it probably extends to any kind of creative work. certainly it can be used in painting: this is exactly what cezanne and klee did.

at y combinator we bet money on it, in the sense that we encourage the startups we fund to work this way. there are always new ideas right under your nose. so look for simple things that other people have overlooked—things people will later claim were "obvious"—especially when they've been led astray by obsolete conventions, or by trying to do things that are superficially impressive. figure out what the real problem is, and make sure you solve that. don't worry about trying to look corporate; the product is what wins in the long term. and launch as soon as you can, so you start learning from users what you should have been making.

reddit is a classic example of this approach. when reddit first launched, it seemed like there was nothing to it. to the graphically unsophisticated its deliberately minimal design seemed like no design at all. but reddit solved the real problem, which was to tell people what was new and otherwise stay out of the way. as a result it became massively successful. now that conventional ideas have caught up with it, it seems obvious. people look at reddit and think the founders were lucky. like all such things, it was harder than it looked. the reddits pushed so hard against the current that they reversed it; now it looks like they're merely floating downstream.

so when you look at something like reddit and think "i wish i could think of an idea like that," remember: ideas like that are all around you. but you ignore them because they look wrong.

05

the other half of "artists ship"

november 2008
paulgraham.com/artistsship.html

forgot to consider the cost. they forgot that companies about to go public are usually rather stretched, and that the weight of a few extra checks that might be easy for general electric to bear are enough to prevent younger companies from being public at all.

once you start to think about the cost of checks, you can start to ask other interesting questions. is the cost increasing or decreasing? is it higher in some areas than others? where does it increase discontinuously? if large organizations started to ask questions like that, they'd learn some frightening things.

i think the cost of checks may actually be increasing. the reason is that software plays an increasingly important role in companies, and the people who write software are particularly harmed by checks.

programmers are unlike many types of workers in that the best ones actually prefer to work hard. this doesn't seem to be the case in most types of work. when i worked in fast food, we didn't prefer the busy times. and when i used to mow lawns, i definitely didn't prefer it when the grass was long after a week of rain.

programmers, though, like it better when they write more code. or more precisely, when they release more code. programmers like to make a difference. good ones, anyway.

for good programmers, one of the best things about working for a startup is that there are few checks on releases. in true startups, there are no external checks at all. if you have an idea for a new feature in the morning, you can write it and push it to the production servers before lunch. and when you can do that, you have more ideas.

at big companies, software has to go through various approvals before it can be launched. and the cost of doing this can be enormous—in fact, discontinuous. i was talking recently to a group of three programmers whose startup had been acquired a few years before by a big company. when they'd been independent, they could release changes instantly. now, they said, the absolute fastest they could get code released on the production servers was two weeks.

this didn't merely make them less productive. it made them hate working for the acquirer.

here's a sign of how much programmers like to be able to work hard: these guys would have paid to be able to release code immediately, the way they used to. i asked them if they'd trade 10% of the acquisition price for the ability to release code immediately, and all three instantly said yes. then i asked what was the maximum percentage of the acquisition price they'd trade for it. they said they didn't want to think about it, because they didn't want to know how high they'd go, but i got the impression it might be as much as half.

they'd have sacrificed hundreds of thousands of dollars, perhaps millions, just to be able to deliver more software to users. and you know what? it would have been perfectly safe to let them. in fact, the acquirer would have been better off; not only wouldn't these guys have broken anything, they'd have gotten a lot more done. so the acquirer is in fact getting worse performance at greater cost. just like the committee approving software purchases.

and just as the greatest danger of being hard to sell to is not that you overpay but that the best suppliers won't even sell to you, the greatest danger of applying too many checks to your programmers is not that you'll make them unproductive, but that good programmers won't even want to work for you.

steve jobs's famous maxim "artists ship" works both ways. artists aren't merely capable of shipping. they insist on it. so if you don't let people ship, you won't have any artists.

06

the hardware renaissance

paulgraham.com/hw.html

07

crazy new ideas

may 2021
paulgraham.com/newideas.html

things more than 10x better. whereas the rewards for attacking new ideas are roughly constant; such attacks seem roughly equally clever regardless of the target.

people will also attack new ideas when they have a vested interest in the old ones. it's not surprising, for example, that some of darwin's harshest critics were churchmen. people build whole careers on some ideas. when someone claims they're false or obsolete, they feel threatened.

the lowest form of dismissal is mere factionalism: to automatically dismiss any idea associated with the opposing faction. the lowest form of all is to dismiss an idea because of who proposed it.

but the main thing that leads reasonable people to dismiss new ideas is the same thing that holds people back from proposing them: the sheer pervasiveness of the current paradigm. it doesn't just affect the way we think; it is the lego blocks we build thoughts out of. popping out of the current paradigm is something only a few people can do. and even they usually have to suppress their intuitions at first, like a pilot flying through cloud who has to trust his instruments over his sense of balance. [4]

paradigms don't just define our present thinking. they also vacuum up the trail of crumbs that led to them, making our standards for new ideas impossibly high. the current paradigm seems so perfect to us, its offspring, that we imagine it must have been accepted completely as soon as it was discovered — that whatever the church thought of the heliocentric model, astronomers must have been convinced as soon as copernicus proposed it. far, in fact, from it. copernicus published the heliocentric model in 1532, but it wasn't till the mid seventeenth century that the balance of scientific opinion shifted in its favor. [5]

few understand how feeble new ideas look when they first appear. so if you want to have new ideas yourself, one of the most valuable things you can do is to learn what they look like when they're born. read about how new ideas happened, and try to get yourself into the heads of people at the time. how did things look to them, when the new idea was only half-finished, and even the person who had it was only half-convinced it was right?

but you don't have to stop at history. you can observe big new ideas being born all around you right now. just look for a reasonable domain expert proposing something that sounds wrong.

if you're nice, as well as wise, you won't merely resist attacking such people, but encourage them. having new ideas is a lonely business. only those who've tried it know how lonely. these people need your help. and if you help them, you'll probably learn something in the process.

---

notes

1 *this domain expertise could be in another field. indeed, such crossovers tend to be particularly promising.*

2 *i'm not claiming this principle extends much beyond math, engineering, and the hard sciences. in politics, for example, crazy-sounding ideas generally are as bad as they sound. though arguably this is not an exception, because the people who propose them are not in fact domain experts; politicians are domain experts in political tactics, like how to get elected and how to get legislation passed, but not in the world that policy acts upon. perhaps no one could be.*

3 *this sense of "paradigm" was defined by thomas kuhn in his structure of scientific revolutions, but i also recommend his copernican revolution, where you can see him at work developing the idea.*

4 *this is one reason people with a touch of asperger's may have an advantage in discovering new ideas. they're always flying on instruments.*

08

heresy

april 2022
paulgraham.com/heresy.html

there are always some heresies — some opinions you'd be punished for expressing. but there are a lot more now than there were a few decades ago, and even those who are happy about this would have to agree that it's so.

why? why has this antiquated-sounding religious concept come back in a secular form? and why now?

you need two ingredients for a wave of intolerance: intolerant people, and an ideology to guide them. the intolerant people are always there. they exist in every sufficiently large society. that's why waves of intolerance can arise so suddenly; all they need is something to set them off.

i've already written an essay describing the aggressively conventional-minded. the short version is that people can be classified in two dimensions according to (1) how independent- or conventional-minded they are, and (2) how aggressive they are about it. the aggressively conventional-minded are the enforcers of orthodoxy.

normally they're only locally visible. they're the grumpy, censorious people in a group — the ones who are always first to complain when something violates the current rules of propriety. but occasionally, like a vector field whose elements become aligned, a large number of aggressively conventional-minded people unite behind some ideology all at once. then they become much more of a problem, because a mob dynamic takes over, where the enthusiasm of each participant is increased by the enthusiasm of the others.

the most notorious 20th century case may have been the cultural revolution. though initiated by mao to undermine his rivals, the cultural revolution was otherwise mostly a grass-roots phenomenon. mao said in essence: there are heretics among us. seek them out and punish them. and that's all the aggressively conventional-minded ever need to hear. they went at it with the delight of dogs chasing squirrels.

to unite the conventional-minded, an ideology must have many of the features of a religion. in particular it must have strict and arbitrary rules that adherents can demonstrate their purity by obeying, and its adherents must believe that anyone who obeys these rules is ipso facto morally superior to anyone who doesn't. [3]

in the late 1980s a new ideology of this type appeared in us universities. it had a very strong component of moral purity, and the aggressively conventional-minded seized upon it with their usual eagerness — all the more because the relaxation of social norms in the preceding decades meant there had been less and less to forbid. the resulting wave of intolerance has been eerily similar in form to the cultural revolution, though fortunately much smaller in magnitude. [4]

i've deliberately avoided mentioning any specific heresies here. partly because one of the universal tactics of heretic hunters, now as in the past, is to accuse those who disapprove of the way in which they suppress ideas of being heretics themselves. indeed, this tactic is so consistent that you could use it as a way of detecting witch hunts in any era.

and that's the second reason i've avoided mentioning any specific heresies. i want this essay to work in the future, not just now. and unfortunately it probably will. the aggressively conventional-minded will always be among us, looking for things to forbid. all they need is an ideology to tell them what. and it's unlikely the current one will be the last.

there are aggressively conventional-minded people on both the right and the left. the reason the current wave of intolerance comes from the left is simply because the new unifying ideology happened to come from the left. the next one might come from the right. imagine what that would be like.

fortunately in western countries the suppression of heresies is nothing like as bad as it used to be. though the window of opinions you can express publicly has narrowed in the last decade, it's still much wider than it was a few hundred years ago. the problem is the derivative. up till about 1985 the window had been

le when they stop bothering to invent pretenses for banning ideas — when, like the medieval church, they say "damn right we're banning ideas, and in fact here's a list of them."

6 *people only have the luxury of ignoring the medical consensus about vaccines because vaccines have worked so well. if we didn't have any vaccines at all, the mortality rate would be so high that most current anti-vaxxers would be begging for them. and the situation with freedom of expression is similar. it's only because they live in a world created by the enlightenment that kids from the suburbs can play at banning ideas.*

thanks *to marc andreessen, chris best, trevor blackwell, nicholas christakis, daniel gackle, jonathan haidt, claire lehmann, jessica livingston, greg lukianoff, robert morris, and garry tan for reading drafts of this.*

09

what i've learned from users

september 2022
paulgraham.com/users.html

often founders know what their problems are, but not their relative importance. [3] they'll come in to talk about three problems they're worrying about. one is of moderate importance, one doesn't matter at all, and one will kill the company if it isn't addressed immediately. it's like watching one of those horror movies where the heroine is deeply upset that her boyfriend cheated on her, and only mildly curious about the door that's mysteriously ajar. you want to say: never mind about your boyfriend, think about that door! fortunately in office hours you can. so while startups still die with some regularity, it's rarely because they wandered into a room containing a murderer. the yc partners can warn them where the murderers are.

not that founders listen. that was another big surprise: how often founders don't listen to us. a couple weeks ago i talked to a partner who had been working for yc for a couple batches and was starting to see the pattern. "they come back a year later," she said, "and say 'we wish we'd listened to you.'"

it took me a long time to figure out why founders don't listen. at first i thought it was mere stubbornness. that's part of the reason, but another and probably more important reason is that so much about startups is counterintuitive. and when you tell someone something counterintuitive, what it sounds to them is wrong. so the reason founders don't listen to us is that they don't believe us. at least not till experience teaches them otherwise. [4]

the reason startups are so counterintuitive is that they're so different from most people's other experiences. no one knows what it's like except those who've done it. which is why yc partners should usually have been founders themselves. but strangely enough, the counterintuitiveness of startups turns out to be another of the things that make yc work. if it weren't counterintuitive, founders wouldn't need our advice about how to do it.

focus is doubly important for early stage startups, because not only do they have a hundred different problems, they don't have anyone to work on them except the founders. if the founders focus on things that don't matter, there's no one focusing on the things that do. so the essence of what happens at yc is to figure out which problems matter most, then cook up ideas for solving them — ideally at a resolution of a week or less — and then try those ideas and measure how well they worked. the focus is on action, with measurable, near-term results.

this doesn't imply that founders should rush forward regardless of the consequences. if you correct course at a high enough frequency, you can be simultaneously decisive at a micro scale and tentative at a macro scale. the result is a somewhat winding path, but executed very rapidly, like the path a running back takes downfield. and in practice there's less backtracking than you might expect. founders usually guess right about which direction to run in, especially if they have someone experienced like a yc partner to bounce their hypotheses off. and when they guess wrong, they notice fast, because they'll talk about the results at office hours the next week. [5]

a small improvement in navigational ability can make you a lot faster, because it has a double effect: the path is shorter, and you can travel faster along it when you're more certain it's the right one. that's where a lot of yc's value lies, in helping founders get an extra increment of focus that lets them move faster. and since moving fast is the essence of a startup, yc in effect makes startups more startup-like.

speed defines startups. focus enables speed. yc improves focus.

why are founders uncertain about what to do? partly because startups almost by definition are doing something new, which means no one knows how to do it yet, or in most cases even what "it" is. partly because startups are so counterintuitive generally. and partly because many founders, especially young and ambitious ones, have been trained to win the wrong way. that took me years to figure out. the educational system in most countries trains you to win by hacking the test instead of actually doing whatever it's supp-

specific startup needs.

2 *when i say the summer 2012 batch was broken, i mean it felt to the partners that something was wrong. things weren't yet so broken that the startups had a worse experience. in fact that batch did unusually well.*

3 *this situation reminds me of the research showing that people are much better at answering questions than they are at judging how accurate their answers are. the two phenomena feel very similar.*

4 *the airbnbs were particularly good at listening — partly because they were flexible and disciplined, but also because they'd had such a rough time during the preceding year. they were ready to listen.*

5 *the optimal unit of decisiveness depends on how long it takes to get results, and that depends on the type of problem you're solving. when you're negotiating with investors, it could be a couple days, whereas if you're building hardware it could be months.*

thanks *to trevor blackwell, jessica livingston, harj taggar, and garry tan for reading drafts of this.*

10

superlinear returns

october 2023
paulgraham.com/superlinear.html

experiences didn't affect customs very much. the emperor was a remote and terrifying figure, not a source of lessons one could use in one's own life.

the most common case of exponential growth in preindustrial times was probably scholarship. the more you know, the easier it is to learn new things. the result, then as now, was that some people were startlingly more knowledgeable than the rest about certain topics. but this didn't affect customs much either. although empires of ideas can overlap and there can thus be far more emperors, in preindustrial times this type of empire had little practical effect. [2]

that has changed in the last few centuries. now the emperors of ideas can design bombs that defeat the emperors of territory. but this phenomenon is still so new that we haven't fully assimilated it. few even of the participants realize they're benefitting from exponential growth or ask what they can learn from other instances of it.

the other source of superlinear returns is embodied in the expression "winner take all." in a sports match the relationship between performance and return is a step function: the winning team gets one win whether they do much better or just slightly better. [3]

the source of the step function is not competition per se, however. it's that there are thresholds in the outcome. you don't need competition to get those. there can be thresholds in situations where you're the only participant, like proving a theorem or hitting a target.

it's remarkable how often a situation with one source of superlinear returns also has the other. crossing thresholds leads to exponential growth: the winning side in a battle usually suffers less damage, which makes them more likely to win in the future. and exponential growth helps you cross thresholds: in a market with network effects, a company that grows fast enough can shut out potential competitors.

fame is an interesting example of a phenomenon that combines both sources of superlinear returns. fame grows exponentially because existing fans bring you new ones. but the fundamental reason it's so concentrated is thresholds: there's only so much room on the a-list in the average person's head.

the most important case combining both sources of superlinear returns may be learning. knowledge grows exponentially, but there are also thresholds in it. learning to ride a bicycle, for example. some of these thresholds are akin to machine tools: once you learn to read, you're able to learn anything else much faster. but the most important thresholds of all are those representing new discoveries. knowledge seems to be fractal in the sense that if you push hard at the boundary of one area of knowledge, you sometimes discover a whole new field. and if you do, you get first crack at all the new discoveries to be made in it. newton did this, and so did durer and darwin.

are there general rules for finding situations with superlinear returns? the most obvious one is to seek work that compounds.

there are two ways work can compound. it can compound directly, in the sense that doing well in one cycle causes you to do better in the next. that happens for example when you're building infrastructure, or growing an audience or brand. or work can compound by teaching you, since learning compounds. this second case is an interesting one because you may feel you're doing badly as it's happening. you may be failing to achieve your immediate goal. but if you're learning a lot, then you're getting exponential growth nonetheless.

this is one reason silicon valley is so tolerant of failure. people in silicon valley aren't blindly tolerant of failure. they'll only continue to bet on you if you're learning from your failures. but if you are, you are in

rs had in the 20th century. there are lots of ambitious projects that don't require much initial funding, and lots of new ways to learn, make money, find colleagues, and reach audiences.

there's still plenty of the old world left, but the rate of change has been dramatic by historical standards. especially considering what's at stake. it's hard to imagine a more fundamental change than one in the returns for performance.

without the damping effect of institutions, there will be more variation in outcomes. which doesn't imply everyone will be better off: people who do well will do even better, but those who do badly will do worse. that's an important point to bear in mind. exposing oneself to superlinear returns is not for everyone. most people will be better off as part of the pool. so who should shoot for superlinear returns? ambitious people of two types: those who know they're so good that they'll be net ahead in a world with higher variation, and those, particularly the young, who can afford to risk trying it to find out. [7]

the switch away from institutions won't simply be an exodus of their current inhabitants. many of the new winners will be people they'd never have let in. so the resulting democratization of opportunity will be both greater and more authentic than any tame intramural version the institutions themselves might have cooked up.

not everyone is happy about this great unlocking of ambition. it threatens some vested interests and contradicts some ideologies. [8] *but if you're an ambitious individual it's good news for you. how should you take advantage of it?*

the most obvious way to take advantage of superlinear returns for performance is by doing exceptionally good work. at the far end of the curve, incremental effort is a bargain. all the more so because there's less competition at the far end — and not just for the obvious reason that it's hard to do something exceptionally well, but also because people find the prospect so intimidating that few even try. which means it's not just a bargain to do exceptional work, but a bargain even to try to.

there are many variables that affect how good your work is, and if you want to be an outlier you need to get nearly all of them right. for example, to do something exceptionally well, you have to be interested in it. mere diligence is not enough. so in a world with superlinear returns, it's even more valuable to know what you're interested in, and to find ways to work on it. [9] *it will also be important to choose work that suits your circumstances. for example, if there's a kind of work that inherently requires a huge expenditure of time and energy, it will be increasingly valuable to do it when you're young and don't yet have children.*

there's a surprising amount of technique to doing great work. it's not just a matter of trying hard. i'm going to take a shot giving a recipe in one paragraph.

choose work you have a natural aptitude for and a deep interest in. develop a habit of working on your own projects; it doesn't matter what they are so long as you find them excitingly ambitious. work as hard as you can without burning out, and this will eventually bring you to one of the frontiers of knowledge. these look smooth from a distance, but up close they're full of gaps. notice and explore such gaps, and if you're lucky one will expand into a whole new field. take as much risk as you can afford; if you're not failing occasionally you're probably being too conservative. seek out the best colleagues. develop good taste and learn from the best examples. be honest, especially with yourself. exercise and eat and sleep well and avoid the more dangerous drugs. when in doubt, follow your curiosity. it never lies, and it knows more than you do about what's worth paying attention to. [10]

and there is of course one other thing you need: to be lucky. luck is always a factor, but it's even more of a factor when you're working on your own rather than as part of an organization. and though there are some valid aphorisms about luck being where preparedness meets opportunity and so on, there's also a

ty sufficiently far, you'll end up in one. your curiosity won't let you be interested in boring questions, and interesting questions tend to create fields with superlinear returns if they're not already part of one.

the territory of superlinear returns is by no means static. indeed, the most extreme returns come from expanding it. so while both ambition and curiosity can get you into this territory, curiosity may be the more powerful of the two. ambition tends to make you climb existing peaks, but if you stick close enough to an interesting enough question, it may grow into a mountain beneath you.

---

notes

there's a limit to how sharply you can distinguish between effort, performance, and return, because they're not sharply distinguished in fact. what counts as return to one person might be performance to another. but though the borders of these concepts are blurry, they're not meaningless. i've tried to write about them as precisely as i could without crossing into error.

1 *evolution itself is probably the most pervasive example of superlinear returns for performance. but this is hard for us to empathize with because we're not the recipients; we're the returns.*

2 *knowledge did of course have a practical effect before the industrial revolution. the development of agriculture changed human life completely. but this kind of change was the result of broad, gradual improvements in technique, not the discoveries of a few exceptionally learned people.*

3 *it's not mathematically correct to describe a step function as superlinear, but a step function starting from zero works like a superlinear function when it describes the reward curve for effort by a rational actor. if it starts at zero then the part before the step is below any linearly increasing return, and the part after the step must be above the necessary return at that point or no one would bother.*

4 *seeking competition could be a good heuristic in the sense that some people find it motivating. it's also somewhat of a guide to promising problems, because it's a sign that other people find them promising. but it's a very imperfect sign: often there's a clamoring crowd chasing some problem, and they all end up being trumped by someone quietly working on another one.*

5 *not always, though. you have to be careful with this rule. when something is popular despite being mediocre, there's often a hidden reason why. perhaps monopoly or regulation make it hard to compete. perhaps customers have bad taste or have broken procedures for deciding what to buy. there are huge swathes of mediocre things that exist for such reasons.*

6 *in my twenties i wanted to be an artist and even went to art school to study painting. mostly because i liked art, but a nontrivial part of my motivation came from the fact that artists seemed least at the mercy of organizations.*

7 *in principle everyone is getting superlinear returns. learning compounds, and everyone learns in the course of their life. but in practice few push this kind of everyday learning to the point where the return curve gets really steep.*

8 *it's unclear exactly what advocates of "equity" mean by it. they seem to disagree among themselves. but whatever they mean is probably at odds with a world in which institutions have less power to control outcomes, and a handful of outliers do much better than everyone else.*

it may seem like bad luck for this concept that it arose at just the moment when the world was shifting in the opposite direction, but i don't think this was a coincidence. i think one reason it arose now is because its adherents feel threatened by rapidly increasing variation in performance.

9 *corollary: parents who pressure their kids to work on something prestigious, like medicine, even though they have no interest in it, will be hosing them even more than they have in the past.*

11

how art can be good

december 2006
paulgraham.com/goodart.html

the picture is slightly more complicated than that, because in the middle of the pond there are overlapping sets of ripples. for example, there might be things that appealed particularly to men, or to people from a certain culture.

if good art is art that interests its audience, then when you talk about art being good, you also have to say for what audience. so is it meaningless to talk about art simply being good or bad? no, because one audience is the set of all possible humans. i think that's the audience people are implicitly talking about when they say a work of art is good: they mean it would engage any human. [4]

and that is a meaningful test, because although, like any everyday concept, "human" is fuzzy around the edges, there are a lot of things practically all humans have in common. in addition to our interest in faces, there's something special about primary colors for nearly all of us, because it's an artifact of the way our eyes work. most humans will also find images of 3d objects engaging, because that also seems to be built into our visual perception. [5] *and beneath that there's edge-finding, which makes images with definite shapes more engaging than mere blur.*

humans have a lot more in common than this, of course. my goal is not to compile a complete list, just to show that there's some solid ground here. people's preferences aren't random. so an artist working on a painting and trying to decide whether to change some part of it doesn't have to think "why bother? i might as well flip a coin." instead he can ask "what would make the painting more interesting to people?" and the reason you can't equal michelangelo by going out and buying a blank canvas is that the ceiling of the sistine chapel is more interesting to people.

a lot of philosophers have had a hard time believing it was possible for there to be objective standards for art. it seemed obvious that beauty, for example, was something that happened in the head of the observer, not something that was a property of objects. it was thus "subjective" rather than "objective." but in fact if you narrow the definition of beauty to something that works a certain way on humans, and you observe how much humans have in common, it turns out to be a property of objects after all. you don't have to choose between something being a property of the subject or the object if subjects all react similarly. being good art is thus a property of objects as much as, say, being toxic to humans is: it's good art if it consistently affects humans in a certain way.

error

so could we figure out what the best art is by taking a vote? after all, if appealing to humans is the test, we should be able to just ask them, right?

well, not quite. for products of nature that might work. i'd be willing to eat the apple the world's population had voted most delicious, and i'd probably be willing to visit the beach they voted most beautiful, but having to look at the painting they voted the best would be a crapshoot.

man-made stuff is different. for one thing, artists, unlike apple trees, often deliberately try to trick us. some tricks are quite subtle. for example, any work of art sets expectations by its level of finish. you don't expect photographic accuracy in something that looks like a quick sketch. so one widely used trick, especially among illustrators, is to intentionally make a painting or drawing look like it was done faster than it was. the average person looks at it and thinks: how amazingly skillful. it's like saying something clever in a conversation as if you'd thought of it on the spur of the moment, when in fact you'd worked it out the day before.

another much less subtle influence is brand. if you go to see the mona lisa, you'll probably be disappointed, because it's hidden behind a thick glass wall and surrounded by a frenzied crowd taking pictures of themselves in front of it. at best you can see it the way you see a friend across the room at a crowded party. the

travel in one direction in time. but if you find a work of art that would appeal equally to your friends, to people in nepal, and to the ancient greeks, you're probably onto something.

my main point here is not how to have good taste, but that there can even be such a thing. and i think i've shown that. there is such a thing as good art. it's art that interests its human audience, and since humans have a lot in common, what interests them is not random. since there's such a thing as good art, there's also such a thing as good taste, which is the ability to recognize it.

if we were talking about the taste of apples, i'd agree that taste is just personal preference. some people like certain kinds of apples and others like other kinds, but how can you say that one is right and the other wrong? [7]

the thing is, art isn't apples. art is man-made. it comes with a lot of cultural baggage, and in addition the people who make it often try to trick us. most people's judgement of art is dominated by these extraneous factors; they're like someone trying to judge the taste of apples in a dish made of equal parts apples and jalapeno peppers. all they're tasting is the peppers. so it turns out you can pick out some people and say that they have better taste than others: they're the ones who actually taste art like apples.

or to put it more prosaically, they're the people who (a) are hard to trick, and (b) don't just like whatever they grew up with. if you could find people who'd eliminated all such influences on their judgement, you'd probably still see variation in what they liked. but because humans have so much in common, you'd also find they agreed on a lot. they'd nearly all prefer the ceiling of the sistine chapel to a blank canvas.

making it

i wrote this essay because i was tired of hearing "taste is subjective" and wanted to kill it once and for all. anyone who makes things knows intuitively that's not true. when you're trying to make art, the temptation to be lazy is as great as in any other kind of work. of course it matters to do a good job. and yet you can see how great a hold "taste is subjective" has even in the art world by how nervous it makes people to talk about art being good or bad. those whose jobs require them to judge art, like curators, mostly resort to euphemisms like "significant" or "important" or (getting dangerously close) "realized." [8]

i don't have any illusions that being able to talk about art being good or bad will cause the people who talk about it to have anything more useful to say. indeed, one of the reasons "taste is subjective" found such a receptive audience is that, historically, the things people have said about good taste have generally been such nonsense.

it's not for the people who talk about art that i want to free the idea of good art, but for those who make it. right now, ambitious kids going to art school run smack into a brick wall. they arrive hoping one day to be as good as the famous artists they've seen in books, and the first thing they learn is that the concept of good has been retired. instead everyone is just supposed to explore their own personal vision. [9]

when i was in art school, we were looking one day at a slide of some great fifteenth century painting, and one of the students asked "why don't artists paint like that now?" the room suddenly got quiet. though rarely asked out loud, this question lurks uncomfortably in the back of every art student's mind. it was as if someone had brought up the topic of lung cancer in a meeting within philip morris.

"well," the professor replied, "we're interested in different questions now." he was a pretty nice guy, but at the time i couldn't help wishing i could send him back to fifteenth century florence to explain in person to leonardo & co. how we had moved beyond their early, limited concept of art. just imagine that conversation.

work with the same energy and honesty that fifteenth century artists did, but art is not one of them.

thanks *to trevor blackwell, jessica livingston, and robert morris for reading drafts of this, and to paul watson for permission to use the image at the top.*

12

how to do what you love

paulgraham.com/love.html

kings, conventional attitudes about work are, without the owners even knowing it, nth-degree imitations of the attitudes of people who've done great things.

what a recipe for alienation. by the time they reach an age to think about what they'd like to do, most kids have been thoroughly misled about the idea of loving one's work. school has trained them to regard work as an unpleasant duty. having a job is said to be even more onerous than schoolwork. and yet all the adults claim to like what they do. you can't blame kids for thinking "i am not like these people; i am not suited to this world."

actually they've been told three lies: the stuff they've been taught to regard as work in school is not real work; grownup work is not (necessarily) worse than schoolwork; and many of the adults around them are lying when they say they like what they do.

the most dangerous liars can be the kids' own parents. if you take a boring job to give your family a high standard of living, as so many people do, you risk infecting your kids with the idea that work is boring. [2] *maybe it would be better for kids in this one case if parents were not so unselfish. a parent who set an example of loving their work might help their kids more than an expensive house.* [3]

it was not till i was in college that the idea of work finally broke free from the idea of making a living. then the important question became not how to make money, but what to work on. ideally these coincided, but some spectacular boundary cases (like einstein in the patent office) proved they weren't identical.

the definition of work was now to make some original contribution to the world, and in the process not to starve. but after the habit of so many years my idea of work still included a large component of pain. work still seemed to require discipline, because only hard problems yielded grand results, and hard problems couldn't literally be fun. surely one had to force oneself to work on them.

if you think something's supposed to hurt, you're less likely to notice if you're doing it wrong. that about sums up my experience of graduate school.

bounds

how much are you supposed to like what you do? unless you know that, you don't know when to stop searching. and if, like most people, you underestimate it, you'll tend to stop searching too early. you'll end up doing something chosen for you by your parents, or the desire to make money, or prestige — or sheer inertia.

here's an upper bound: do what you love doesn't mean, do what you would like to do most this second. even einstein probably had moments when he wanted to have a cup of coffee, but told himself he ought to finish what he was working on first.

it used to perplex me when i read about people who liked what they did so much that there was nothing they'd rather do. there didn't seem to be any sort of work i liked that much. if i had a choice of (a) spending the next hour working on something or (b) be teleported to rome and spend the next hour wandering about, was there any sort of work i'd prefer? honestly, no.

but the fact is, almost anyone would rather, at any given moment, float about in the caribbean, or have sex, or eat some delicious food, than work on hard problems. the rule about doing what you love assumes a certain length of time. it doesn't mean, do what will make you happiest this second, but what will make you happiest over some longer period, like a week or a month.

unproductive pleasures pall eventually. after a while you get tired of lying on the beach. if you want to stay happy, you have to do something.

the other big force leading people astray is money. money by itself is not that dangerous. when something pays well but is regarded with contempt, like telemarketing, or prostitution, or personal injury litigation, ambitious people aren't tempted by it. that kind of work ends up being done by people who are "just trying to make a living." (tip: avoid any field whose practitioners say this.) the danger is when money is combined with prestige, as in, say, corporate law, or medicine. a comparatively safe and prosperous career with some automatic baseline prestige is dangerously tempting to someone young, who hasn't thought much about what they really like.

the test of whether people love what they do is whether they'd do it even if they weren't paid for it — even if they had to work at another job to make a living. how many corporate lawyers would do their current work if they had to do it for free, in their spare time, and take day jobs as waiters to support themselves?

this test is especially helpful in deciding between different kinds of academic work, because fields vary greatly in this respect. most good mathematicians would work on math even if there were no jobs as math professors, whereas in the departments at the other end of the spectrum, the availability of teaching jobs is the driver: people would rather be english professors than work in ad agencies, and publishing papers is the way you compete for such jobs. math would happen without math departments, but it is the existence of english majors, and therefore jobs teaching them, that calls into being all those thousands of dreary papers about gender and identity in the novels of conrad. no one does that kind of thing for fun.

the advice of parents will tend to err on the side of money. it seems safe to say there are more undergrads who want to be novelists and whose parents want them to be doctors than who want to be doctors and whose parents want them to be novelists. the kids think their parents are "materialistic." not necessarily. all parents tend to be more conservative for their kids than they would for themselves, simply because, as parents, they share risks more than rewards. if your eight year old son decides to climb a tall tree, or your teenage daughter decides to date the local bad boy, you won't get a share in the excitement, but if your son falls, or your daughter gets pregnant, you'll have to deal with the consequences.

discipline

with such powerful forces leading us astray, it's not surprising we find it so hard to discover what we like to work on. most people are doomed in childhood by accepting the axiom that work = pain. those who escape this are nearly all lured onto the rocks by prestige or money. how many even discover something they love to work on? a few hundred thousand, perhaps, out of billions.

it's hard to find work you love; it must be, if so few do. so don't underestimate this task. and don't feel bad if you haven't succeeded yet. in fact, if you admit to yourself that you're discontented, you're a step ahead of most people, who are still in denial. if you're surrounded by colleagues who claim to enjoy work that you find contemptible, odds are they're lying to themselves. not necessarily, but probably.

although doing great work takes less discipline than people think — because the way to do great work is to find something you like so much that you don't have to force yourself to do it — finding work you love does usually require discipline. some people are lucky enough to know what they want to do when they're 12, and just glide along as if they were on railroad tracks. but this seems the exception. more often people who do great things have careers with the trajectory of a ping-pong ball. they go to school to study a, drop out and get a job doing b, and then become famous for c after taking it up on the side.

sometimes jumping from one sort of work to another is a sign of energy, and sometimes it's a sign of laziness. are you dropping out, or boldly carving a new path? you often can't tell yourself. plenty of people who will later do great things seem to be disappointments early on, when they're trying to find their niche.

among projects. the disadvantage of this route is that it's slow and uncertain. even tenure is not real freedom.

the two-job route has several variants depending on how long you work for money at a time. at one extreme is the "day job," where you work regular hours at one job to make money, and work on what you love in your spare time. at the other extreme you work at something till you make enough not to have to work for money again.

the two-job route is less common than the organic route, because it requires a deliberate choice. it's also more dangerous. life tends to get more expensive as you get older, so it's easy to get sucked into working longer than you expected at the money job. worse still, anything you work on changes you. if you work too long on tedious stuff, it will rot your brain. and the best paying jobs are most dangerous, because they require your full attention.

the advantage of the two-job route is that it lets you jump over obstacles. the landscape of possible jobs isn't flat; there are walls of varying heights between different kinds of work. [7] the trick of maximizing the parts of your job that you like can get you from architecture to product design, but not, probably, to music. if you make money doing one thing and then work on another, you have more freedom of choice.

which route should you take? that depends on how sure you are of what you want to do, how good you are at taking orders, how much risk you can stand, and the odds that anyone will pay (in your lifetime) for what you want to do. if you're sure of the general area you want to work in and it's something people are likely to pay you for, then you should probably take the organic route. but if you don't know what you want to work on, or don't like to take orders, you may want to take the two-job route, if you can stand the risk.

don't decide too soon. kids who know early what they want to do seem impressive, as if they got the answer to some math question before the other kids. they have an answer, certainly, but odds are it's wrong.

a friend of mine who is a quite successful doctor complains constantly about her job. when people applying to medical school ask her for advice, she wants to shake them and yell "don't do it!" (but she never does.) how did she get into this fix? in high school she already wanted to be a doctor. and she is so ambitious and determined that she overcame every obstacle along the way — including, unfortunately, not liking it.

now she has a life chosen for her by a high-school kid.

when you're young, you're given the impression that you'll get enough information to make each choice before you need to make it. but this is certainly not so with work. when you're deciding what to do, you have to operate on ridiculously incomplete information. even in college you get little idea what various types of work are like. at best you may have a couple internships, but not all jobs offer internships, and those that do don't teach you much more about the work than being a batboy teaches you about playing baseball.

in the design of lives, as in the design of most other things, you get better results if you use flexible media. so unless you're fairly sure what you want to do, your best bet may be to choose a type of work that could turn into either an organic or two-job career. that was probably part of the reason i chose computers. you can be a professor, or make a lot of money, or morph it into any number of other kinds of work.

it's also wise, early on, to seek jobs that let you do many different things, so you can learn faster what various kinds of work are like. conversely, the extreme version of the two-job route is dangerous because it teaches you so little about what you like. if you work hard at being a bond trader for ten years, thinking that you'll quit and write novels when you have enough money, what happens when you quit and then discover that you don't actually like writing novels?